

-
6. Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"

int main()
{
    char buffer[100];

    mkfifo(FIFO1, 0666);
    mkfifo(FIFO2, 0666);

    printf("Server waiting for client...\n");

    int fd1 = open(FIFO1, O_RDONLY);
    int fd2 = open(FIFO2, O_WRONLY);

    while (1)
    {
        read(fd1, buffer, sizeof(buffer));

        if (strcmp(buffer, "exit") == 0)
            break;

        printf("Client: %s\n", buffer);

        printf(buffer, "Server received: %s", buffer);
        write(fd2, buffer, strlen(buffer) + 1);
    }

    close(fd1);
    close(fd2);
}
```

```
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define FIF01 "client_to_server"
#define FIF02 "server_to_client"

int main()
{
    char buffer[100];

    mkfifo(FIF01, 0666);
    mkfifo(FIF02, 0666);

    printf("Server waiting for client...\n");

    int fd1 = open(FIF01, O_RDONLY);
    int fd2 = open(FIF02, O_WRONLY);

    while (1)
    {
        read(fd1, buffer, sizeof(buffer));

        if (strcmp(buffer, "exit") == 0)
            break;

        printf("Client: %s\n", buffer);

        printf(buffer, "Server received: %s", buffer);
        write(fd2, buffer, strlen(buffer) + 1);
    }

    close(fd1);
    close(fd2);

    unlink(FIF01);
    unlink(FIF02);

    return 0;
}
venkatesh@LAPTOP-LL9RQ023:~$ |
```

```
venkatesh@LAPTOP-LL9RQ023:~$ cat week6.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define FIF01 "client_to_server"
#define FIF02 "server_to_client"

int main()
{
    char buffer[100];

    mkfifo(FIF01, 0666);
    mkfifo(FIF02, 0666);

    printf("Server waiting for client...\n");

    int fd1 = open(FIF01, O_RDONLY);
    int fd2 = open(FIF02, O_WRONLY);

    while (1)
    {
        read(fd1, buffer, sizeof(buffer));

        if (strcmp(buffer, "exit") == 0)
            break;

        printf("Client: %s\n", buffer);

        printf(buffer, "Server received: %s", buffer);
        write(fd2, buffer, strlen(buffer) + 1);
    }

    close(fd1);
    close(fd2);

    unlink(FIF01);
    unlink(FIF02);
}
```

```

    return 0;
}
venkatesh@LAPTOP-LL9RQ023:~$ gcc week6b.c
venkatesh@LAPTOP-LL9RQ023:~$ ./a.out
Enter message: 1
1
Enter message: ^C
venkatesh@LAPTOP-LL9RQ023:~$ gcc week6.c
venkatesh@LAPTOP-LL9RQ023:~$ ./a.out
Enter message: ^C
venkatesh@LAPTOP-LL9RQ023:~$ nano week6.c
venkatesh@LAPTOP-LL9RQ023:~$ rm week6.c
venkatesh@LAPTOP-LL9RQ023:~$ nano week6.c
venkatesh@LAPTOP-LL9RQ023:~$ gcc week6.c
week6.c: In function 'main':
week6.c:32:43: warning: 'sprintf' directive writing up to 99 bytes into a region of size 83 [-Wformat-overflow=]
   32 |         sprintf(buffer, "Server received: %s", buffer);
      |
week6.c:32:9: note: 'sprintf' output between 18 and 117 bytes into a destination of size 100
   32 |         sprintf(buffer, "Server received: %s", buffer);
      |
venkatesh@LAPTOP-LL9RQ023:~$ nano week6.c
venkatesh@LAPTOP-LL9RQ023:~$ gcc week6.c
venkatesh@LAPTOP-LL9RQ023:~$ ./a.out
Server waiting for client...
Client: Venkatesh

```

```

Enter message: ^C
venkatesh@LAPTOP-LL9RQ023:~$ nano week6b.c
venkatesh@LAPTOP-LL9RQ023:~$ rm week6b.c
venkatesh@LAPTOP-LL9RQ023:~$ nano week6b.c
venkatesh@LAPTOP-LL9RQ023:~$ gcc week6b.c
venkatesh@LAPTOP-LL9RQ023:~$ ./a.out
Enter message: Venkatesh
Venkatesh
Enter message: |

```

GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define FIF01 "client_to_server"
#define FIF02 "server_to_client"

int main()
{
    char buffer[100];

    int fd1 = open(FIF01, O_WRONLY);
    int fd2 = open(FIF02, O_RDONLY);

    while (1)
    {
        printf("Enter message: ");
        fgets(buffer, sizeof(buffer), stdin);
        buffer[strcspn(buffer, "\n")] = '\0';

        write(fd1, buffer, strlen(buffer) + 1);

        if (strcmp(buffer, "exit") == 0)
            break;

        read(fd2, buffer, sizeof(buffer));
        printf("%s\n", buffer);
    }

    close(fd1);
    close(fd2);

    return 0;
}
```